

LABORATÓRIO DE PROGRAMAÇÃO FULL STACK

Contrato Inicial — API de Chamados

Atividade Hands On — Semana 02/04

Aluno(a):	Richardson, Flavio, Nilson
Turma:	
Professor(a):	Ralph
Data:	_21_ / _08_ / 2026

Documento elaborado de forma simples e objetiva, seguindo os itens solicitados na atividade.

1. Cenário e objetivo

A empresa está iniciando uma aplicação web para gerenciar chamados de suporte interno. O front-end precisa saber como listar, registrar e acompanhar os chamados, enquanto o back-end ficará responsável pela API e, depois, pela persistência dos dados.

Por isso, este documento funciona como um contrato inicial da API: ele define, de forma clara, como as equipes deverão se comunicar, evitando que cada uma faça suposições diferentes durante o desenvolvimento.

2. Recurso

Nome do recurso: chamados

Finalidade: registrar e acompanhar solicitações de suporte feitas por usuários.

Em outras palavras, um chamado representa um problema, uma dúvida ou uma solicitação que precisa ser analisada pela equipe de suporte.

3. Formato dos dados

As requisições e respostas da API usarão o formato JSON. Nas operações que enviam dados, será utilizado o cabeçalho:

```
Content-Type: application/json
```

4. Atributos do chamado

Campo	Tipo	Obrigatório na criação?	Descrição	Exemplo
id	número inteiro	Não	Identificador único do chamado	42
título	texto	Sim	Resumo do problema	Tela sem acesso
descricao	texto	Sim	Detalhamento do problema	Não consigo acessar a tela de consultas.
prioridade	texto	Sim	Nível de prioridade	alta
status	texto	Não	Situação atual do chamado	aberto

O campo id será gerado pelo sistema. O campo status também não precisa ser enviado na criação; quando um chamado for criado, o valor inicial será aberto.

4.1 Valores aceitos

Para evitar valores diferentes representando a mesma coisa, serão usados os seguintes padrões:

- Prioridade: baixa, media ou alta.
- Status: aberto, em_andamento ou concluído.

5. Endpoints, métodos e parâmetros

As URIs representam recursos e usam nomes no plural. As ações ficam por conta dos métodos HTTP, então não serão criadas rotas como /criarChamado ou /excluirChamado.

Operação	Método	URI	Parâmetros	Status de sucesso
Listar chamados	GET	/chamados	status opcional	200
Consultar chamado	GET	/chamados/{id}	id na URI	200
Criar chamado	POST	/chamados	corpo JSON	201
Atualizar chamado	PATCH	/chamados/{id}	id + corpo JSON	200
Remover chamado	DELETE	/chamados/{id}	id na URI	204

6. Exemplos de requisição e resposta

6.1 Criação de chamado — exemplo completo

Requisição:

```
POST /chamados
Content-Type: application/json

{
  "titulo": "Tela sem acesso",
  "descricao": "Não consigo acessar a tela de consultas.",
  "prioridade": "alta"
}
```

Resposta de sucesso:

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /chamados/42

{
  "id": 42,
  "titulo": "Tela sem acesso",
  "descricao": "Não consigo acessar a tela de consultas.",
  "prioridade": "alta",
  "status": "aberto"
}
```

O código 201 indica que o chamado foi criado. O cabeçalho Location mostra onde o novo recurso pode ser consultado.

6.2 Listar chamados

```
GET /chamados
```

Exemplo de resposta:

```
HTTP/1.1 200 OK
Content-Type: application/json

[
  {
    "id": 42,
    "titulo": "Tela sem acesso",
    "descricao": "Não consigo acessar a tela de consultas.",
    "prioridade": "alta",
    "status": "aberto"
  }
]
```

6.3 Filtrar chamados por status

O parâmetro de consulta status é opcional. Exemplo:

```
GET /chamados?status=aberto
```

6.4 Consultar um chamado

```
GET /chamados/42

HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 42,
  "titulo": "Tela sem acesso",
  "descricao": "Não consigo acessar a tela de consultas.",
  "prioridade": "alta",
  "status": "aberto"
}
```

6.5 Atualizar parcialmente um chamado

O PATCH foi escolhido porque permite alterar somente os campos necessários.

```
PATCH /chamados/42
Content-Type: application/json

{
  "status": "em andamento"
}
```

Resposta:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 42,
  "titulo": "Tela sem acesso",
  "descricao": "Não consigo acessar a tela de consultas.",
  "prioridade": "alta",
  "status": "em_andamento"
}
```

6.6 Remover um chamado

```
DELETE /chamados/42

HTTP/1.1 204 No Content
```

O status 204 informa que a exclusão foi realizada com sucesso e que não há conteúdo para retornar no corpo da resposta.

7. Cenários de erro

7.1 Erro 400 — dado inválido

Exemplo: tentativa de criar um chamado sem o campo obrigatório titulo.

```
POST /chamados
Content-Type: application/json

{
  "descricao": "Não consigo acessar a tela de consultas.",
  "prioridade": "alta"
}
```

Resposta:

```
HTTP/1.1 400 Bad Request
Content-Type: application/json

{
  "erro": "Dado inválido",
  "detalhes": [
    {
      "campo": "título",
      "mensagem": "O título é obrigatório."
    }
  ]
}
```

7.2 Erro 404 — recurso inexistente

Exemplo: consulta a um identificador que não existe.

```
GET /chamados/9999

HTTP/1.1 404 Not Found
Content-Type: application/json

{
  "erro": "Chamado não encontrado",
  "detalhes": [
    {
      "campo": "id",
      "mensagem": "Não existe chamado com o identificador 9999."
    }
  ]
}
```

8. Códigos de status previstos

Código	Significado	Uso no contrato
200 OK	Operação realizada com sucesso	Listar, consultar e atualizar
201 Created	Novo recurso criado	Criação de chamado
204 No Content	Operação concluída sem conteúdo	Remoção de chamado
400 Bad Request	Dados enviados são inválidos	Campo obrigatório ausente ou valor inválido
404 Not Found	Recurso não encontrado	Chamado inexistente

9. Decisões técnicas tomadas

1. Requisições e respostas serão representadas em JSON.
2. A URI do recurso será /chamados, no plural.
3. O id será gerado pelo sistema.
4. Todo chamado novo começará com status aberto.
5. As prioridades aceitas serão baixa, media e alta.
6. Os status aceitos serão aberto, em_andamento e concluido.
7. PATCH será utilizado para atualização parcial.
8. A listagem poderá receber o parâmetro opcional status.
9. As mensagens de erro terão uma estrutura padronizada e compreensível para quem implementar a interface.

10. Dúvidas pendentes

Como este é um contrato inicial, algumas informações podem ser avaliadas em versões futuras:

- Será necessário registrar quem abriu o chamado (por exemplo, usuario_id)?

- Será necessário registrar quem é o responsável pelo atendimento (responsavel_id)?
- Será necessário incluir data_criacao e data_atualizacao?

Esses campos não foram incluídos agora para manter o primeiro contrato simples e focado apenas no que a atividade solicita.

11. Revisão por outra equipe

A atividade pede uma revisão por pares. A equipe revisora deve verificar se as URIs representam recursos, se os métodos HTTP estão coerentes, se os JSONs são válidos, se os códigos de status fazem sentido e se o documento é suficiente para implementar um consumidor da API.

Modelo de registro para preencher após a revisão real em sala:

Item	Registro
Equipe revisora	_____
Ajuste realizado	_____ _____
Dúvida apontada	_____ _____

Sugestão de ajuste técnico já considerado no contrato: deixar explícitos os valores permitidos para prioridade e status, evitando que front-end e back-end usem nomes diferentes para a mesma informação.

12. Checklist final

- ☒ O recurso possui nome, finalidade e atributos definidos.
- ☒ As coleções usam URI no plural, como /chamados.
- ☒ Um item individual é identificado por /chamados/{id}.
- ☒ Cada operação possui método HTTP coerente.
- ☒ Há exemplo de POST com corpo em JSON.
- ☒ Há exemplo de resposta 201 para criação bem-sucedida.
- ☒ Há cenário de erro 400 para dado inválido.
- ☒ Há cenário de erro 404 para recurso inexistente.
- ☒ Os exemplos usam os mesmos nomes de campos.
- ☒ Decisões e dúvidas pendentes foram registradas.
- ☐ Outra equipe revisou o contrato (marcar após a revisão real).

13. Conclusão

Este contrato define de forma simples como o recurso chamados será utilizado. Com ele, a equipe de front-end consegue entender como listar, consultar, criar, atualizar e remover chamados, enquanto a equipe de back-end possui uma referência clara para implementar a API.

Também foram definidos o formato JSON, os parâmetros, os principais códigos HTTP e uma estrutura consistente para respostas de erro. Assim, as equipes podem trabalhar de forma alinhada desde o início do desenvolvimento.